Guía de Laboratorio N°2

Métodos mágicos · @property · Herencia · super()

Docente: Camilo Alejandro Fuentes Beals – PhD
Programación Científica

Instrucciones Generales

Antes de comenzar:

- Todos los ejercicios deben resolverse en Python 3.10 o superior.
- Cada clase debe incluir los métodos solicitados; no se aceptan funciones independientes.
- Incluye comentarios explicativos en el código donde se indique.
- Puedes consultar la sintaxis de referencia al final de esta guía.
- Entrega por Github.

Ejercicio 1: Clase Producto — Métodos Mágicos

En este ejercicio diseñarás una clase **Producto** que modele un artículo de una tienda. El objetivo es practicar los métodos mágicos de representación, comparación y aritmética.

Parte A — Estructura base (8 pts)

Crea la clase **Producto** con los siguientes atributos de instancia:

- nombre (str): nombre del producto
- precio (float): precio en pesos
- stock (int): unidades disponibles

Implementa los siguientes métodos:

a1) __str__: retorna una cadena legible, ej.: Arroz – $1.200 (50 unidades)
a2) __repr__: retorna la representación técnica, ej.: Producto('Arroz', 1200, 50)
a3) __init__: constructor con los tres atributos

Parte B — Comparación (9 pts)

Agrega los siguientes métodos de comparación basados en el **precio**:

b1) __eq__(self, otro): dos productos son iguales si tienen el mismo precio
b2) __lt__(self, otro): un producto es menor si su precio es menor
b3) Usa @total_ordering de functools para derivar >, >=, <=

Parte C — Aritmética (8 pts)

c1) __add__(self, otro): suma los stocks de dos productos del mismo nombre, retorna un nuevo Producto
c2) __mul__(self, n): multiplica el precio por n, retorna un nuevo Producto (útil para aplicar descuentos)

Prueba tu solución

```
p1 = Producto('Arroz', 1200, 50)
p2 = Producto('Arroz', 900, 30)
p3 = Producto('Leche', 1200, 20)

print(p1)              # Arroz – $1200 (50 unidades)
print(repr(p2))        # Producto('Arroz', 900, 30)
print(p1 == p3)        # True  → mismo precio
print(p2 < p1)         # True  → 900 < 1200
print(sorted([p1, p2, p3]))  # ordenado por precio

combinado = p1 + p2    # stock = 80
print(combinado)       # Arroz – $1200 (80 unidades)

oferta = p1 * 0.8      # precio con 20% descuento
print(oferta)          # Arroz – $960.0 (50 unidades)
```

Ejercicio 2: Clase CuentaBancaria — @property y Validación

Diseñarás una clase **CuentaBancaria** que use **@property** y setters para garantizar que el saldo nunca sea negativo y que el titular sea válido.

Parte A — Atributos con validación (12 pts)

La clase debe tener los atributos **_titular** y **_saldo** como privados. Implementa:

- **a1)** titular (property): getter que retorna el nombre del titular
- **a2)** titular (setter): valida que el nombre tenga al menos 3 caracteres, si no lanza ValueError
- **a3)** saldo (property): getter que retorna el saldo actual
- **a4)** saldo (setter): valida que el saldo no sea negativo, si no lanza ValueError con mensaje claro

Parte B — Métodos de operación (7 pts)

- **b1)** depositar(monto): agrega monto al saldo (valida que monto > 0)
- **b2)** retirar(monto): resta monto del saldo (valida que no deje saldo negativo)

Parte C — Representación (6 pts)

- **c1)** __str__: ej: Cuenta de Ana López | Saldo: $45.000
- **c2)** __repr__: ej: CuentaBancaria(titular='Ana López', saldo=45000)
- **c3)** Propiedad calculada aprobada (bool): True si saldo >= 10000

Prueba tu solución

```
cuenta = CuentaBancaria('Ana López', 50000)
print(cuenta)              # Cuenta de Ana López | Saldo: $50000

cuenta.depositar(20000)
print(cuenta.saldo)        # 70000

cuenta.retirar(15000)
print(cuenta)              # Cuenta de Ana López | Saldo: $55000

# Validaciones
try:
    cuenta.saldo = -1000   # debe lanzar ValueError
except ValueError as e:
    print(e)               # Saldo no puede ser negativo

try:
    CuentaBancaria('AB', 0) # titular muy corto
except ValueError as e:
    print(e)

print(cuenta.aprobada)     # True
```

Ejercicio 3: Herencia — Figuras Geométricas

Construirás una jerarquía de clases para figuras geométricas usando herencia y **super()**. La clase base definirá la interfaz y las subclases implementarán el comportamiento específico.

Parte A — Clase base Figura (8 pts)

- Atributo: color (str)
- __init__(self, color): inicializa el color
- area(self): lanza NotImplementedError (método abstracto)
- perimetro(self): lanza NotImplementedError (método abstracto)
- __str__: ej: Figura roja | Área: 78.54 | Perímetro: 31.42
- __repr__: representación técnica completa

Parte B — Subclase Circulo (9 pts)

Hereda de **Figura**. Usa import math.

b1) __init__(self, color, radio): usa super() para inicializar color, luego asigna radio con validación (> 0)
b2) area(self): retorna math.pi * radio²
b3) perimetro(self): retorna 2 * math.pi * radio

Parte C — Subclase Rectangulo (8 pts)

c1) __init__(self, color, ancho, alto): usa super(), valida que ancho y alto > 0
c2) area(self): retorna ancho * alto

c3) perimetro(self): retorna 2 * (ancho + alto)
c4) es_cuadrado (property): True si ancho == alto

Prueba tu solución

```
import math

c = Circulo('rojo', 5)
r = Rectangulo('azul', 4, 6)
cuad = Rectangulo('verde', 5, 5)

print(c)               # Figura roja | Área: 78.54 | Perímetro: 31.42
print(r)               # Figura azul | Área: 24 | Perímetro: 20

print(cuad.es_cuadrado)   # True
print(r.es_cuadrado)      # False

# Polimorfismo
figuras = [c, r, cuad]
for f in figuras:
    print(f'Área: {f.area():.2f}')

# Ordenar por área (requiere __lt__)
# Bonus: agrega __lt__ basado en área y ordénalas
```

Ejercicio 4: Integración — Inventario con Métodos de Secuencia — 25 pts

Este ejercicio integra todo lo visto: diseñarás una clase **Inventario** que actúe como un contenedor nativo de Python, implementando métodos de secuencia y usando como elementos objetos **Producto** del Ejercicio 1.

Parte A — Métodos de secuencia (15 pts)

La clase **Inventario** debe contener una lista interna **_productos** e implementar:

a1) agregar(producto): agrega un Producto a la lista interna
a2) __len__: retorna el número de productos
a3) __getitem__(self, i): permite acceso por índice inv[0]
a4) __contains__(self, producto): permite usar in operator
a5) __iter__: permite iterar con for p in inv

Parte B — Representación y análisis (10 pts)

b1) __str__: lista los productos, uno por línea con su representación __str__
b2) __repr__: Inventario(n_productos=X)
b3) total_valor (property): suma precio * stock de todos los productos
b4) mas_caro (property): retorna el Producto con mayor precio usando max()
b5) buscar(nombre) -> list: retorna todos los productos cuyo nombre contenga la cadena buscada (case-insensitive)

Prueba tu solución

```
inv = Inventario()
inv.agregar(Producto('Arroz', 1200, 50))
inv.agregar(Producto('Leche', 950, 30))
inv.agregar(Producto('Pan', 1800, 100))
inv.agregar(Producto('Panetón', 3500, 10))

print(len(inv))            # 4
print(inv[0])              # Arroz – $1200 (50 unidades)

p_buscar = Producto('Leche', 950, 30)
print(p_buscar in inv)     # True

for p in inv:
    print(p)

print(inv.total_valor)     # 1200*50 + 950*30 + 1800*100 + 3500*10 = ...
print(inv.mas_caro)        # Panetón – $3500 (10 unidades)

resultados = inv.buscar('pan')
for r in resultados:
    print(r)               # Pan y Panetón
```

Rubrica de Evaluación

Ejercicio 1 — Clase Producto: Métodos Mágicos				25 pts
Criterio	**Excelente**	**Satisfactorio**	**Parcial**	**Insuficiente**
Parte A __init__, __str__, __repr__ (8 pts)	Implementa los 3 métodos correctamente. __str__ es legible para usuario. __repr__ permite recrear el objeto. (8 pts)	Implementa los 3 métodos con errores menores de formato o retorno. (6 pts)	Implementa 1–2 métodos. Alguno incompleto o con lógica incorrecta. (4 pts)	No implementa o los métodos no funcionan. (0–2 pts)
Parte B __eq__, __lt__, @total_ordering (9 pts)	Implementa __eq__ y __lt__ correctamente. Usa @total_ordering. Los 6 operadores funcionan. (9 pts)	Implementa __eq__ y __lt__ pero no usa @total_ordering o hay errores en la comparación. (6–7 pts)	Solo implementa uno de los dos métodos requeridos. (4–5 pts)	No implementa comparadores o la lógica es incorrecta. (0–2 pts)
Parte C __add__, __mul__ (8 pts)	Ambos retornan un nuevo objeto Producto. __add__ suma stocks correctamente. __mul__ ajusta precio sin mutar el original. (8 pts)	Ambos métodos presentes pero uno tiene error en retorno o modifica el objeto original. (6 pts)	Solo uno de los dos métodos implementado y funcional. (4 pts)	No implementa o los métodos no retornan objetos correctos. (0–2 pts)
Puntaje Ejercicio 1				**/ 25**

Ejercicio 2 — Clase CuentaBancaria: @property y Validación				25 pts
Criterio	**Excelente**	**Satisfactorio**	**Parcial**	**Insuficiente**
Parte A @property + setters (12 pts)	Getter y setter para titular y saldo. Setter valida correctamente (ValueError con mensaje). Atributos privados con prefijo _. Self.valor en __init__ usa el setter. (12 pts)	Getters y setters presentes pero: - Falta mensaje en ValueError, o - No usa setter en __init__, o - Prefijo _ incorrecto en uno. (8–10 pts)	Solo un property implementado correctamente. El otro no valida o no es property. (5–7 pts)	No usa @property. Atributos públicos sin validación. (0–3 pts)
Parte B depositar, retirar (7 pts)	Ambos métodos presentes. depositar valida monto > 0. retirar valida que no quede saldo negativo. Ambos usan el setter (no asignan directamente). (7 pts)	Ambos métodos presentes pero: - Falta una de las validaciones, o - Asignan directamente sin pasar por setter. (5–6 pts)	Solo uno de los métodos implementado o ambos sin validaciones. (3–4 pts)	Métodos ausentes o no modifican el saldo. (0–1 pts)
Parte C __str__, __repr__, aprobada (6 pts)	__str__ legible con titular y saldo formateado. __repr__ reproducible. Property aprobada retorna bool correcto. (6 pts)	Dos de los tres elementos correctos o los tres con errores menores. (4–5 pts)	Solo uno correcto o el formato es incorrecto en varios. (2–3 pts)	No implementa representación o la property falta. (0–1 pts)
Puntaje Ejercicio 2				**/ 25**

Ejercicio 3 — Herencia: Figuras Geométricas — 25 pts

Criterio	Excelente	Satisfactorio	Parcial	Insuficiente
Parte A Clase base Figura (8 pts)	Figura tiene __init__(color), area() y perimetro() con NotImplementedError. __str__ formateado correctamente. __repr__ técnico presente. (8 pts)	Clase base presente pero: - Falta NotImplementedError en uno, o - __str__ o __repr__ con errores de formato. (6 pts)	Solo define __init__ y uno de los métodos abstractos. Representaciones ausentes. (3–4 pts)	Figura no definida o no tiene estructura de clase base. (0–1 pts)
Parte B Clase Circulo (9 pts)	Hereda de Figura con sintaxis correcta. Usa super().__init__(color). Valida radio > 0. area() y perimetro() calculan correctamente con math.pi. (9 pts)	Hereda correctamente y usa super() pero: - Falta validación de radio, o - Error menor en fórmula de area o perimetro. (6–7 pts)	Define la clase pero no usa super() o no hereda correctamente. Fórmulas presentes pero incorrectas. (4–5 pts)	No hereda de Figura o los métodos no están implementados. (0–2 pts)
Parte C Clase Rectangulo + es_cuadrado (8 pts)	Hereda de Figura, usa super(). Valida ancho y alto > 0. area() y perimetro() correctos. Property es_cuadrado retorna bool correcto. (8 pts)	Clase presente y funcional pero: - Falta validación o es_cuadrado es método regular en vez de property. (6 pts)	Hereda pero sin super(). Fórmulas presentes pero error en es_cuadrado. (3–4 pts)	No hereda correctamente o métodos ausentes. (0–2 pts)

Puntaje Ejercicio 3 — **/ 25**

Ejercicio 4 — Integración: Clase Inventario				25 pts
Criterio	**Excelente**	**Satisfactorio**	**Parcial**	**Insuficiente**
Parte A Métodos de secuencia (15 pts)	Implementa agregar, __len__, __getitem__, __contains__ e __iter__ correctamente. Todos funcionan de forma nativa (len(), obj[i], in, for). (15 pts)	4 de los 5 métodos correctos o los 5 con errores menores que no afectan funcionamiento principal. (11–12 pts)	2–3 métodos implementados correctamente. Los restantes ausentes o con errores de lógica. (7–9 pts)	Solo 1 método o ninguno implementado. La clase no actúa como contenedor. (0–4 pts)
Parte B Representación + análisis (10 pts)	__str__ lista todos los productos. __repr__ muestra cantidad correcta. total_valor, mas_caro y buscar funcionan correctamente. buscar es case-insensitive. (10 pts)	3–4 de los 5 elementos correctos. buscar presente pero no es case-insensitive, o mas_caro falla con lista vacía. (7–8 pts)	1–2 elementos correctos. __str__ básico pero total_valor o mas_caro ausentes. (4–5 pts)	Solo representación mínima o ninguno. No implementa análisis. (0–2 pts)
Puntaje Ejercicio 4				**/ 25**